

Issue: Result Identity Drift vs Data Corruption

The failure at hand is not a simple data corruption but a canonicalization mismatch. The producer's reported *semantic identity* (hash) matches its own recomputation, but the consumer's recomputed hash differs. In other words, the **same apparent JSON content yields different digests** when processed by producer and consumer. This points to a discrepancy in the *canonicalization or identity function* used on each side, not to random corruption. In effect:

Producer/Independent: $H(x) = 2648b7...$

Consumer: $H(x) = 219d65...$

This is analogous to known “signature mismatch” scenarios in distributed systems. For example, payment protocols warn that an invalid signature often means the *signer and verifier used different JSON canonicalization* (e.g. different key ordering or whitespace) [14†L1-L4] .

In our case, the code paths touching `ResultSchema` and receipts likely involve (a) constructing a canonical JSON form of the result to compute its hash, and (b) verifying it on the consumer side. The independent recomputation matching the producer indicates that *the producer's canonicalization function is consistent with itself*; the consumer's mismatch shows it must be using a different canon. This exactly mirrors scenarios addressed by canonical-JSON standards.

JSON Canonicalization Standards

To avoid such drift, **JSON Canonicalization Schemes** exist. The IETF draft **JCS (JSON Canonicalization Scheme)** defines a deterministic way to serialize JSON so that any conformant producer and consumer will produce identical byte sequences (hence identical hashes) [21†L146-L154] [13†L39-L47] . JCS mandates, for example, strict `JSON.stringify`-style formatting, elimination of whitespace, and *sorted object keys* [21†L146-L154] . In fact, the JCS spec notes:

“Producers and consumers...generate consistent results. This document describes the JSON Canonicalization Scheme (JCS)... by using deterministic property sorting.” [21†L146-L154] [13†L39-L47] .

Practically, differences often arise from things like key order, numeric formatting, or optional fields. The JCS draft lists constraints (e.g. I-JSON subset, no duplicate keys, no whitespace) that ensure a “hashable representation” [21†L146-L154] [13†L39-L47] . If the producer used one canonicalizer (e.g. some `JSON.stringify` with certain options) and the consumer another (or default stringification), their hashes will diverge even for semantically identical objects.

The observed failure strongly suggests this: *the consumer's canonicalizer apparently includes/excludes or orders fields differently*. A minimal test might insert trivial differences (e.g. extra whitespace or reordered

fields) to see if the hash changes on one side. In payment protocols (e.g. AP2 mandates), the advice is clear: when you see a hash/signature mismatch, *use a shared canonicalizer on both sides* 【14†L1-L4】.

Binding Identities: Canonical Requests and Results

Architecturally, this issue reveals a missing linkage between **request contract** and **result identity**. Industry best practices (e.g. the Jacobian project) recommend including a *canonical request digest* and invocation ID in every result 【18†L244-L251】. As that proposal notes:

“The request digest is computed from the canonical JSON encoding of the validated request input. The invocation ID is a stable...identifier that uniquely identifies this invocation.” 【18†L244-L251】.

By hashing the *request* in a canonical form, the result can be unambiguously tied to its input. In our scenario, the proclaimed result contract (`ResultSchema`) should likewise be part of the identity. The failure occurs because the consumer “rebuilds” the identity without access to the same contract or canonical logic. If `ResultSchema` (and any relevant context) were included in the identity function, consumer and producer could use the same inputs to derive the same hash.

This ties into general principles: many secure logging and attestation schemes insist the verifier **independently recompute** identifiers and reject mismatches. For instance, the QIK-VRT draft states that a consumer “*MUST independently rederive the result. A mismatch MUST prevent ordinary release.*” 【9†L336-L344】. That is, no trusted admission should occur if the hashes disagree.

Practical Testing and Distinction Preservation

To reproduce the issue, one would run the producer path to get a result and its `Receipt.ResultSchema`. Then simulate the consumer’s admission logic on that output. If the consumer rejects it, the difference lies in the hash functions. A targeted test could vary only the JSON formatting:

- **Minimal Assay A:** Take a correct output, reserialize it (e.g. pretty vs compact) and see if the consumer still rejects.
- **Assay B:** Keep all semantic fields the same but reorder object keys, or remove/add trivial empty fields, and compare hashes on both sides.

If any such harmless reformatting changes the consumer’s hash, we know exactly what to adjust. In effect, we need to force the system to answer:

“Can two JSON documents that are semantically equivalent but differ in ordering/format lead to different admissibility?”

The consumer’s procedure should treat such differences as identical (collapse them) for consequence, not as failure. To achieve that, they must share a single canonical form. A true equivalence class of worlds is defined by the requested consequence and admissibility predicate; only differences that can change the admitted consequence need to be preserved. Hence, *fields irrelevant to admissibility (like indentation, key order, or even schema fields if already satisfied) should be collapsed safely.*

Map to Patent Claims and Evidence

The patent-claim insight from earlier messages is that the **bound identity function** is crucial. Prior-art in structured-output validation mostly covers *syntax/schema validation*. For example, many systems validate JSON against a declared schema after generation (as in [22]). But our specimen shows a deeper mechanism: the *declared contract (ResultSchema) must be part of the identity function that governs admissible consequences*. In other words:

- **Claimed Mechanism:** The runtime should include the canonical `ResultSchema` in the identity of the candidate output (just like a canonical request digest). The **admission predicate** should require that the actual output adheres to that exact contract.
- **Observed Evidence:** Currently, the code's `FindingAdmission` loses the `ResultSchema` identity. A malformed or differently-serialized output can slip through if schema fields aren't rechecked. The experiment shows that two distinct valid JSON (6-key vs 8-key) both passed completion under the 9-key contract because the contract wasn't enforced on output. This proves *the schema identity was not bound to the result admission*.
- **Negative Control:** A low-capacity run (64k) yielded truncated JSON and was correctly rejected (syntax fail), while higher-capacity runs produced *different* valid JSON shapes yet were both admitted. Thus, admission was effectively checking only "valid JSON" not "correct schema" ($\text{Accept}(r) \approx \text{JSONValid}(r)$).
- **Gap & Suggestion:** The patent-worthy fix is to treat the **schema contract as part of the consequence identity**: i.e. `Admit(r) iff Parse(r)  $\wedge$  Conforms(r, Contract_j)`. And ensure that the *identity in receipts* includes the actual contract used. This is analogous to binding request identity in [18†L244-L251].

In summary, resolving this issue requires treating the schema/contract as a first-class part of the identity function and admission check. Practically, the system must (a) agree on one canonical JSON form across producer and consumer, and (b) verify that the output matches the *exact* declared schema, not just any valid JSON.

References

- JSON Canonicalization (JCS) – IETF draft defining how producers/consumers achieve *identical* serialized JSON for hashing [21†L146-L154] [13†L39-L47] .
- AI agent payments guide – notes that "canonical-JSON drift between signer and verifier" (key order, whitespace) causes signature errors [14†L1-L4] .
- Jacobian GitHub issue – recommends including a *canonical request digest* in every result identity [18†L244-L251] .
- QIK-VRT draft – mandates that the consumer "*MUST independently rederive the result. A mismatch MUST prevent ordinary release.*" [9†L336-L344] (illustrating the need to compare hashes on both sides).